DESENVOLVIMENTO MOBILE · MÓDULO 14 · TRILHA KOTLIN · NÍVEL INTERMEDIÁRIO

Testes e Qualidade

Pirâmide de testes, testes unitários com coroutines, fakes e mocks, testes de UI em Compose, desempenho, acessibilidade e integração contínua

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Planejar testes segundo a pirâmide de testes.
- Testar funções, casos de uso e ViewModels com JUnit, coroutines e Turbine.
- Usar *fakes* e *mocks* para isolar dependências.
- Escrever testes de interface em Compose e testes de banco com Room.
- Diagnosticar problemas de desempenho e recomposição.
- Tornar o app acessível e automatizar a verificação com integração contínua.

Roteiro

Neste módulo

- Fundamentos de teste de software
- Testes no Android: visão geral
- Testes unitários
- Testes de integração e de UI
- Desempenho
- Acessibilidade
- Qualidade contínua
- Fechamento

Fundamentos de teste de software

Erro, defeito e falha

Três palavras que não são sinônimos

Erro (*error*): engano cometido por uma pessoa. **Defeito** (*defect*, *bug*): imperfeição no código ou em outro artefato, consequência do erro. **Falha** (*failure*): comportamento incorreto observado quando o defeito é executado.



Consequência

Um defeito pode ficar anos sem causar falha, se a linha nunca for executada nas condições certas. Testar é **provocar** essas condições.

Um pouco de história

Em 1947, a equipe do computador Harvard Mark II, da qual Grace Hopper fazia parte, achou uma mariposa presa num relé e a colou no diário de bordo. A palavra *bug* já existia, mas o episódio a popularizou.

Para que serve testar

Objetivos dos testes

- Encontrar defeitos antes do usuário.
- Dar confiança para entregar.
- Impedir que defeitos corrigidos voltem (**regressão**).
- Documentar, em código executável, como o sistema deve se comportar.
- Permitir **refatorar** sem medo.

Princípio fundamental

Testes mostram que defeitos **existem**; nunca provam que **não existem**. A ideia é de Edsger Dijkstra.

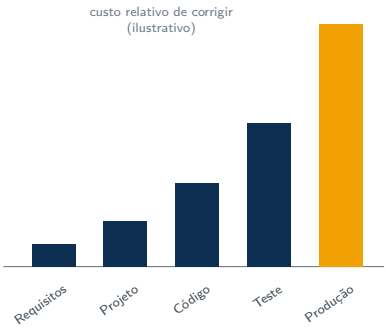
Teste exaustivo é impossível

Uma função com dois inteiros de 32 bits tem 2^{64} combinações de entrada, cerca de $1{,}8 \times 10^{19}$. A um bilhão de testes por segundo, seriam quase 600 anos. Por isso escolhemos casos **representativos** com técnicas de projeto de testes.

Analogia

O *crash test* de um carro não bate o veículo em todas as velocidades e ângulos: escolhe poucos cenários que representam os acidentes reais.

O custo de achar tarde



Por que custa mais

Um defeito achado em produção exige investigar relatos, reproduzir, corrigir, testar de novo, gerar versão, passar pela revisão da loja e esperar os usuários atualizarem. No app, ainda há dados já gravados errado.

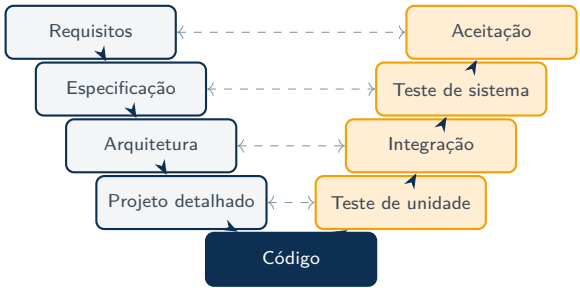
Shift left

Antecipar a qualidade: revisar requisitos, escrever testes junto com o código e rodá-los a cada *commit*.

Um pouco de história

Barry Boehm mediu esse efeito nos anos 1970 e 1980; os números variam entre projetos, mas a tendência se repete.

Verificação, validação e níveis de teste



Modelo em V: cada nível de teste confere um nível de especificação.

Verificação

"Estamos construindo o produto **corretamente**?" Confere o software contra a especificação.

Validação

"Estamos construindo o produto **certo**?" Confere se o software resolve o problema real do usuário.

Dica

O teste de aceitação é feito com o cliente: no LabMonitor, os técnicos do laboratório usando o app.

Caixa-preta: partições e valores-limite

Teste caixa-preta

Projetado a partir da **especificação**, sem olhar o código: entradas e saídas esperadas.

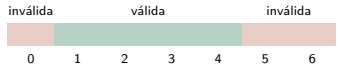
Partição de equivalência

Divide as entradas em classes que o sistema deve tratar do mesmo jeito; basta um representante de cada classe.

Análise de valor-limite

Defeitos se concentram nas bordas (< no lugar de <=). Teste o limite e seus vizinhos imediatos.

Regra: a reserva dura de 1 a 4 horas inteiras



Casos	Por quê
2 h	representante da classe válida
0 h e 1 h	limite inferior e vizinho
4 h e 5 h	limite superior e vizinho
−1 h, “abc”	entradas absurdas

Caixa-branca: critérios de cobertura

Teste caixa-branca

Projetado a partir da **estrutura interna** do código: comandos, decisões e caminhos.

Pseudocódigo

```
se temp >= limite E alertaAtivo:
    notificar()
registrar(temp)
```

Atenção

Cobertura mede o que foi **executado**, não o que foi **conferido**: um teste sem `expect` também "cobre".

Critério	Exige, no exemplo
Comandos	cada linha rodar ao menos uma vez: 1 teste (30 °C, ativo) basta
Decisões	cada `se` (*branch*) ser verdadeiro e falso: 2 testes
Condições	cada condição simples verdadeira e falsa: `temp` e `alertaAtivo` variando
Caminhos	todas as rotas possíveis; cresce exponencialmente com laços e `se`s

Hierarquia

Caminhos ⇒ decisões ⇒ comandos: quem satisfaz um critério mais forte satisfaz os mais fracos.

Dublês de teste

Dublê	O que é	Exemplo no LabMonitor
Dummy	objeto só para preencher um parâmetro; nunca é usado	um *logger* vazio exigido pelo construtor
Stub	devolve respostas prontas	repositório que sempre devolve 3 bancadas
Spy	stub que também registra as chamadas	conta quantas notificações foram enviadas
Mock	objeto com **expectativas** de interação, verificadas no teste	"`reservar(7)` deve ser chamado uma vez"
Fake	implementação funcional simplificada	banco em memória, API falsa com dados fixos

Dublê de teste

Objeto que substitui uma dependência real (rede, banco, sensor, relógio) durante o teste.

Analogia

Como o dublê de cinema: entra nas cenas perigosas ou caras no lugar do ator.

Um pouco de história

A classificação é de Gerard Meszaros, no livro *xUnit Test Patterns* (2007).

Propriedades de um bom teste

FIRST	Significado
Fast	rápido: roda em milissegundos, então roda sempre
Independent	não depende da ordem nem de outros testes
Repeatable	mesmo resultado em qualquer máquina e horário
Self-validating	passa ou falha sozinho, sem alguém olhar a saída
Timely	escrito junto com o código (ou antes)

Estrutura

AAA: *Arrange*, *Act*, *Assert*.
BDD: *Given*, *When*, *Then* (dado, quando, então).

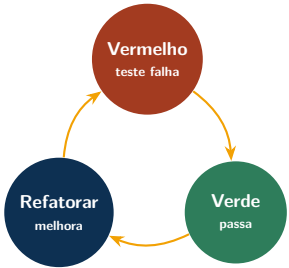
Teste instável (*flaky*)

Passa e falha sem o código mudar. Causas típicas: depender do relógio real, da rede, da ordem de execução, de esperas fixas (*sleep*) ou de estado compartilhado entre testes.

Teste também é código

Merece nomes claros, nada de duplicação e revisão. Um bom nome de teste descreve uma **regra**: "reserva sobreposta é recusada".

TDD, BDD e regressão



TDD

Test-Driven Development (Kent Beck): o teste é escrito **antes** do código e guia o projeto.

BDD: cenário em linguagem natural (Gherkin)

```
Funcionalidade: Reserva de bancada
  Cenário: horário já ocupado
    Dado que a bancada B7 está reservada das 14h às 16h
    Quando eu peço a B7 das 15h às 17h
    Então vejo "Horário indisponível"
```

BDD

Behavior-Driven Development: descreve o comportamento em exemplos que clientes, testadores e devs entendem; os cenários viram testes.

Teste de regressão

Rodar de novo os testes existentes após cada mudança, para garantir que nada que funcionava quebrou. Automatizar é o que torna isso viável.

Desafios do teste em apps móveis

Desafio	O que pode dar errado
Fragmentação	milhares de modelos Android, versões de sistema, tamanhos e densidades de tela
Rede	sem sinal, 3G lento, troca Wi-Fi/móvel no meio de uma requisição
Interrupções	ligação, notificação, app enviado ao fundo e encerrado pelo sistema
Hardware	câmera, GPS, Bluetooth e sensores diferentes ou ausentes
Permissões	usuário nega, concede "só desta vez" ou revoga depois
Recursos	bateria fraca, pouca memória, armazenamento cheio
Pessoas	idioma, fonte grande, leitor de tela, modo escuro

Estratégia

- Automatizar a lógica em testes de unidade e widget (rápidos, sem aparelho).
- Definir uma **matriz de dispositivos** representativa (versões antigas e novas, tela pequena e grande).
- Usar emuladores no dia a dia e aparelhos reais ou *device farms* antes de publicar.
- Simular rede ruim e interrupções.

Dica

As estatísticas do Android Studio e do console da loja mostram quais versões seus usuários realmente usam.

Qualidade é mais do que testar

Modelo de qualidade ISO/IEC 25010

Organiza a qualidade de um produto de software em características como adequação funcional, eficiência de desempenho, compatibilidade, usabilidade, confiabilidade, segurança, manutenibilidade e portabilidade.

Dica

A revisão de 2023 renomeou algumas características (usabilidade virou capacidade de interação) e incluiu *safety*.

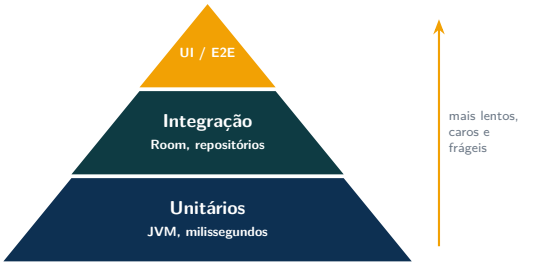
Técnicas que se somam

- **Estáticas** (sem executar): revisão de código, análise estática, *lint*, tipagem forte.
- **Dinâmicas** (executando): testes automatizados e exploratórios.
- **Em produção**: monitoramento de falhas, métricas e *feedback* dos usuários.

Qualidade se **constrói** durante o desenvolvimento; testes confirmam que ela está lá.

Testes no Android: visão geral

A pirâmide de testes



- **Unitários** (`src/test`): muitos, rápidos, rodam no computador.
- **Integração**: várias peças juntas (DAO + banco em memória).
- **UI / ponta a ponta** (`src/androidTest`): poucos, verificam fluxos críticos no aparelho.

O que testar primeiro

Regras de negócio (casos de uso), transformações de dados e os ViewModels. É onde estão os erros mais caros.

Arquitetura testável

As camadas, as interfaces e a injeção de dependência dos Módulos 08–10 existem justamente para permitir isso.

Testes unitários

Primeiro teste: funções puras

src/test/.../ConversoesTest.kt

```
class ConversoesTest {

  @Test
  fun `zero celsius é 32 fahrenheit`() {
    assertEquals(32.0, 0.0.cParaF(), 0.001)
  }

  @Test
  fun `adc de 12 bits no fundo de escala vale vref`() {
    assertEquals(3.3, 4095.adcParaVolts(), 0.001)
  }

  @Test
  fun `leitura fora da faixa gera alerta`() {
    val alertas = VerificadorFaixa(10.0..40.0)
      .verificar(listOf(25.0, 41.2, 9.0))
    assertEquals(listOf(41.2, 9.0), alertas)
  }
}
```

Estrutura AAA

Arrange (prepare os dados), **Act** (execute) e **Assert** (verifique). Um comportamento por teste.

Nomes legíveis

Kotlin aceita nomes de função entre crases: o relatório de testes vira uma especificação em português.

Executando

```
./gradlew test
# relatório em app/build/reports/tests/
```

Testando coroutines

MainDispatcherRule.kt

```
// Substitui Dispatchers.Main (inexistente na JVM)
class MainDispatcherRule(
  val dispatcher: TestDispatcher =
    UnconfinedTestDispatcher()
) : TestWatcher() {
  override fun starting(d: Description) =
    Dispatchers.setMain(dispatcher)
  override fun finished(d: Description) =
    Dispatchers.resetMain()
}
```

Teste com runTest

```
@Test
fun `sincronizar tenta de novo após falha`() = runTest {
  val api = FakeApi(falhasAntesDeResponder = 2)
  val repo = BancadaRepositoryImpl(FakeDao(), api)
  repo.sincronizarComRetry()   // delays são "pulados"
  assertEquals(3, api.chamadas)
}
```

runTest

Executa código `suspend` em testes com **tempo virtual**: um `delay(60_000)` termina instantaneamente.

Dispatchers injetados

Repositórios que recebem o dispatcher no construtor (Módulo 06) podem receber um `StandardTestDispatcher` no teste.

Dependência

`kotlinx-coroutines-test` em `testImplementation`.

Fakes *versus* mocks

Fake: implementação simples

```
class FakeBancadaRepository : BancadaRepository {
  private val dados =
    MutableStateFlow<List<Bancada>>(emptyList())
  var falharSync = false

  fun emitir(lista: List<Bancada>) { dados.value = lista
      }

  override fun bancadas() = dados
  override suspend fun sincronizar() {
    if (falharSync) throw IOException("sem rede")
  }
  override suspend fun reservar(id: Int, ate: Instant) {}
}
```

Mock com MockK

```
val api = mockk<LabApi>()
coEvery { api.listarBancadas() } throws IOException()

val repo = BancadaRepositoryImpl(FakeDao(), api)
val r = runCatching { repo.sincronizar() }

assertTrue(r.isFailure)
coVerify(exactly = 1) { api.listarBancadas() }
```

Quando usar

Fake: reutilizável, comportamento realista; preferido pelo Google. **Mock**: verificar *interações* (foi chamado? quantas vezes?).

Testando o ViewModel com Turbine

PainelViewModelTest.kt

```
class PainelViewModelTest {
  @get:Rule val main = MainDispatcherRule()
  private val repo = FakeBancadaRepository()
  private val vm = PainelViewModel(repo)

  @Test
  fun `filtro mostra apenas bancadas correspondentes`() =
    runTest {
      repo.emitir(listOf(Bancada(1, "Eletrônica", 24.0),
                         Bancada(2, "Máquinas", 22.0)))
      vm.uiState.test {
        assertEquals(2, awaitItem().bancadas.size)
        vm.onFiltroChange("Máq")
        val estado = awaitItem()
        assertEquals(1, estado.bancadas.size)
        assertEquals("Máquinas", estado.bancadas[0].titulo)
        cancelAndIgnoreRemainingEvents()
      }
    }
}
```

Turbine

Biblioteca da Cash App que coleta um `Flow` e permite verificar cada emissão em sequência com `awaitItem()`.

Dica

Teste o **estado** que o ViewModel expõe, não seus detalhes internos. Assim o teste sobrevive a refatorações.

Atenção

`stateIn(WhileSubscribed)` só calcula com alguém coletando. O `test {}` do Turbine é esse coletor.

Testes de integração e de UI

Room com banco em memória

src/androidTest/.../BancadaDaoTest.kt

```
@RunWith(AndroidJUnit4::class)
class BancadaDaoTest {
  private lateinit var db: AppDatabase
  private lateinit var dao: BancadaDao

  @Before fun criar() {
    db = Room.inMemoryDatabaseBuilder(
      ApplicationProvider.getApplicationContext(),
      AppDatabase::class.java).build()
    dao = db.bancadaDao()
  }
  @After fun fechar() = db.close()

  @Test fun salvarEObservar() = runTest {
    dao.salvar(BancadaEntity(1, "B1", "L-12"))
    val lista = dao.observarTodas().first()
    assertEquals("B1", lista.single().nome)
  }
}
```

Banco em memória

Criado do zero a cada teste e descartado no fim: testes independentes e rápidos, sem afetar dados reais.

Migrações

`MigrationTestHelper` cria o banco na versão antiga, aplica a migração e valida o esquema resultante.

Dica

Os testes em `androidTest` rodam com `./gradlew connectedAndroidTest` em um aparelho ou emulador.

Testes de interface em Compose

TelaPainelTest.kt

```
class TelaPainelTest {
  @get:Rule val compose = createComposeRule()

  @Test fun mostraAlertaQuandoAcimaDoLimite() {
    compose.setContent {
      CartaoBancada(numero = 3, temperatura = 35.0)
    }
    compose.onNodeWithText("Acima do limite!")
      .assertIsDisplayed()
  }

  @Test fun clicarNaBancadaChamaCallback() {
    var aberta = -1
    compose.setContent {
      TelaPainel(estadoExemplo, {}, {}, onAbrir = { aberta = it
      })
    }
    compose.onNodeWithTag("bancada-2").performClick()
    assertEquals(2, aberta)
  }
}
```

Encontrar, agir, verificar

Encontrar: `onNodeWithText`, `onNodeWithTag`, `onNodeWithContentDescription`.
Agir: `performClick`, `performTextInput`.
Verificar: `assertIsDisplayed`, `assertExists`.

Telas sem estado

A divisão *Route*/*Screen* do Módulo 08 permite testar a tela passando qualquer `UiState`, sem ViewModel nem rede.

Desempenho

Diagnosticando desempenho

Ferramentas

- **Profiler** do Android Studio: CPU, memória, energia; identifica *jank* (quadros perdidos).
- **Layout Inspector**: contagem de recomposições por composable.
- **Macrobenchmark**: mede inicialização e rolagem de forma reproduzível.
- **Android vitals** (Play Console): ANRs e falhas nos aparelhos reais.

Atenção

Meça sempre em build **release** e em aparelho real: o build debug é muito mais lento.

Compose rápido

- Nada de trabalho pesado durante a composição: formate e filtre no ViewModel.
- `key` estável em listas *lazy*.
- `derivedStateOf` para estados que mudam com frequência (ex.: posição de rolagem).
- Classes de estado imutáveis (`val`, `List`).

Baseline Profiles

Arquivo que lista o código usado na inicialização. A Play Store pré-compila esse código e o app abre bem mais rápido.

Acessibilidade

Apps para todas as pessoas

Semântica em Compose

```
// imagem informativa: descreva
Icon(Icons.Default.Warning,
  contentDescription = "Temperatura acima do limite")

// card inteiro lido como um único item
Card(Modifier.semantics(mergeDescendants = true) {}) {
  Text("Bancada 3"); Text("24,8 °C")
}

// estado anunciado pelo leitor de tela
Switch(checked = ligado, onCheckedChange = { ligado = it
      },
  modifier = Modifier.semantics {
    stateDescription = if (ligado) "ligado" else
      "desligado"
  })
```

Checklist

- `contentDescription` em ícones e imagens informativas.
- Alvos de toque de pelo menos **48 dp**.
- Contraste mínimo de **4,5:1** para texto.
- Funciona com fonte em **200 %** (use `sp`).
- Informação não depende só de cor (verde/vermelho + ícone ou texto).
- Navegável com **TalkBack** e por teclado.

Dica

O app **Accessibility Scanner** analisa a tela aberta e aponta problemas automaticamente.

Qualidade contínua

Análise estática e integração contínua

.github/workflows/android.yml

```yaml
name: Android CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 17 }
      - uses: gradle/actions/setup-gradle@v4
      - run: ./gradlew lint testDebugUnitTest assembleDebug
      - uses: actions/upload-artifact@v4
        with:
          name: apk-debug
          path: app/build/outputs/apk/debug/*.apk
```

Ferramenta	Verifica
Android Lint	APIs incorretas, permissões, recursos não usados, acessibilidade
ktlint	formatação e estilo Kotlin
detekt	complexidade e más práticas
Testes	comportamento

Integração contínua

A cada *push*, um servidor compila, analisa e testa o projeto. Um *pull request* só é aceito com tudo verde.

Dica

Cheque as versões atuais das *actions* no GitHub Marketplace antes de usar.

Fechamento

Resumo

Resumo

- Muitos testes unitários, alguns de integração e poucos de UI.
- `runTest` + `MainDispatcherRule` para coroutines; Turbine para `Flow`.
- Fakes para simular dependências; mocks (MockK) para verificar interações.
- Room em memória e `createComposeRule` para integração e interface.
- Desempenho: meça em release, no aparelho, com Profiler, Layout Inspector e Baseline Profiles.
- Acessibilidade e lint fazem parte da qualidade; a CI verifica tudo a cada *push*.

Glossário (1/2)

Termo	Significado
Teste de unidade	verifica uma unidade isolada (função, classe, `ViewModel`) sem dependências reais
Teste instrumentado	roda no aparelho ou no emulador (`src/androidTest`)
Teste de UI em Compose	monta composables e interage com eles pela árvore de semântica
Teste de regressão	reexecução dos testes após mudanças para garantir que nada quebrou
Dublê de teste	substituto de uma dependência real: *dummy*, *stub*, *spy*, *mock* ou *fake*
Fake	implementação simplificada e funcional (repositório em memória)
Mock	dublê que verifica as interações esperadas (chamadas e argumentos)
`runTest`	executa coroutines em teste com tempo virtual
Turbine	biblioteca que coleta e confere os valores emitidos por um `Flow`

Glossário (2/2)

Termo	Significado
`StandardTestDispatcher`	*dispatcher* controlado pelo teste, com ordem previsível
`ComposeTestRule`	monta a tela, encontra nós e simula toques no teste de UI
Cobertura	proporção do código executada pelos testes (comandos, decisões, condições)
Análise estática	inspeção do código sem executá-lo: Android Lint, detekt, ktlint
Teste instável (*flaky*)	teste que passa ou falha sem mudança no código
Macrobenchmark	mede abertura e rolagem do app real, de forma reproduzível
Baseline Profile	lista do código crítico, pré-compilado para o app abrir mais rápido
CI	integração contínua: compilar e testar automaticamente a cada *push*

Questões de revisão

1. Diferencie erro, defeito e falha usando um exemplo do LabMonitor.
2. Por que testes não provam a ausência de defeitos? Que técnicas reduzem o número de casos sem perder poder de detecção?
3. A regra é "alerta quando a temperatura for maior ou igual a 30 °C". Quais valores você testaria e por quê?
4. Mostre um exemplo em que 100% de cobertura de comandos não implica 100% de cobertura de decisões.
5. Quando usar um *fake* e quando usar um *mock*?
6. Por que usar `runTest` e um *dispatcher* de teste para testar código com `delay`?
7. O que um teste de UI em Compose verifica, e por que ele encontra os elementos pela semântica?
8. Cite três causas de testes instáveis e como evitá-las.

Exercícios

Exercício 1 — Unitários

Escreva testes para `VerificarAlertasUseCase` cobrindo: lista vazia, todas abaixo do limite, uma acima, temperatura nula e mudança do limite.

Exercício 2 — ViewModel

Teste o `PainelViewModel` para os estados carregando, sucesso, vazio e erro de sincronização usando `FakeBancadaRepository` e Turbine.

Exercício 3 — Acessibilidade

Rode o Accessibility Scanner nas telas do LabMonitor, navegue com TalkBack e corrija ao menos três problemas encontrados.

LabMonitor — Etapa 14

Atinja ao menos 70 % de cobertura nos casos de uso e ViewModels, escreva dois testes de UI para os fluxos de reserva e configure o GitHub Actions para rodar lint e testes.

Para aprofundar

- Testes no Android: `https://developer.android.com/training/testing`
- Testando coroutines: `https://developer.android.com/kotlin/coroutines/test`
- Testes em Compose: `https://developer.android.com/develop/ui/compose/testing`
- Turbine: `https://github.com/cashapp/turbine` MockK: `https://mockk.io`
- Desempenho em Compose: `https://developer.android.com/develop/ui/compose/performance`
- Baseline Profiles: `https://developer.android.com/topic/performance/baselineprofiles`
- Acessibilidade: `https://developer.android.com/guide/topics/ui/accessibility`

Respostas das questões de revisão

1. **Erro**: o dev escreve > em vez de >=. **Defeito**: a regra de alerta fica errada no código. **Falha**: a exatamente 30 °C o alerta não aparece.
2. O espaço de entradas é grande demais para testar tudo; um teste só mostra o comportamento nos casos executados. Partição de equivalência e valores-limite escolhem casos representativos.
3. 29,9 (logo abaixo, sem alerta), 30,0 (no limite, com alerta) e 30,1 (acima), além de um valor típico de cada classe e entradas inválidas (sem leitura).
4. `if (cond) x()` seguido de `y()`: um teste com `cond` verdadeira executa todas as linhas, mas o lado falso da decisão nunca é exercitado.
5. *Fake* quando importa o resultado (dados devolvidos); *mock* quando importa a interação (se e como uma dependência foi chamada).
6. Com tempo real, o teste seria lento e instável. O `runTest` usa tempo virtual, e um `delay(5_000)` passa na hora; o *dispatcher* de teste (com `Dispatchers.setMain`) deixa a ordem de execução previsível.
7. O que a tela mostra e como reage a toques e digitação, sem depender de pixels. Ela busca nós por texto, `contentDescription` ou `testTag`: a mesma árvore usada pelos leitores de tela.
8. Relógio real, rede real e esperas fixas; estado compartilhado entre testes; animações. Evite com *fakes*, tempo virtual, `waitUntil` em vez de `Thread.sleep` e estado recriado a cada teste.

Fim do Módulo 14

Próximo: Módulo 15 — Publicação na Google Play: assinatura, AAB, Play Console, testes fechados, LGPD e CI/CD.